

Connect to a PostgreSQL Database Server

1. Introduction

To begin using PostgreSQL, you must first connect to the database server. The PostgreSQL installer provides **two main client tools** for connection:

- **pgAdmin** – Graphical User Interface (GUI, web-based).
- **SQL Shell (psql)** – Terminal-based interactive shell.

Once connected, you can create databases, manage tables, and run SQL commands.

2. Key Concepts

Term	Description
pgAdmin	GUI tool for managing PostgreSQL databases visually.
psql	Command-line tool for executing SQL queries.
Server	PostgreSQL engine running in the background.
Database	Organized collection of data. Default database: postgres.
User	Account to log in. Default user: postgres.
Port	Communication channel for PostgreSQL. Default: 5432.

Password	Set during installation; required for authentication.
-----------------	---

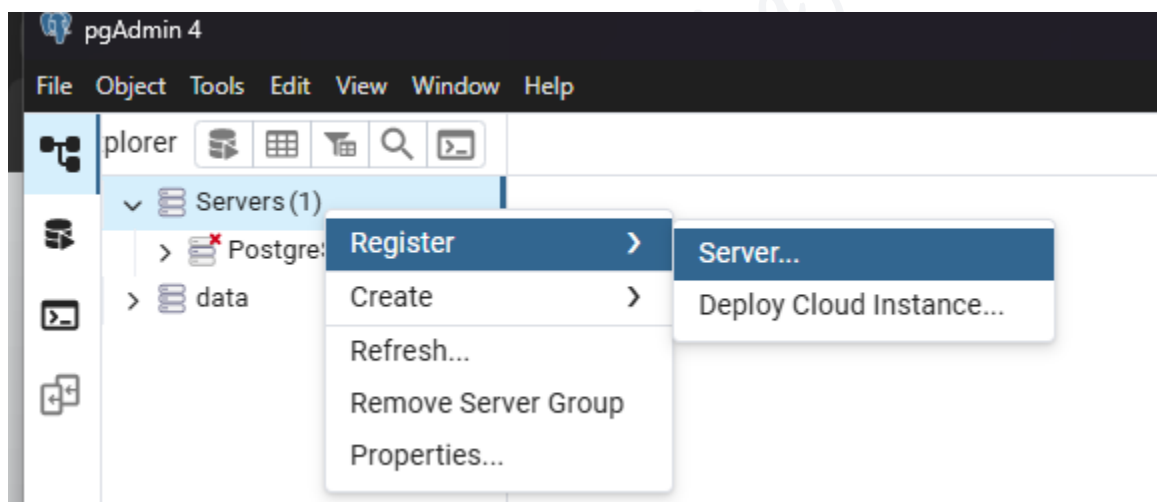
3. Connecting with pgAdmin (GUI Tool)

Step 1: Launch pgAdmin

- Search for pgAdmin in your system and open it.
- pgAdmin will open in your browser.

Step 2: Create a Server

- Right-click on **Servers** → **Create** → **Server**.



Step 3: Enter Server Name

- Enter a descriptive name (e.g., PostgreSQL1, akshay).

Register - Server

General | Connection | Parameters | SSH Tunnel | Advanced | Post Connection SQL | Tags

Name: PostgreSQL1

Server group: Servers

Background: ☐

Foreground: ☐

Connect now?: ☒

Comments:

Either Host name or Service must be specified.

Close Reset Save

Step 4: Provide Connection Details

- Host: localhost
- Port: 5432
- Username: postgres
- Password: (your installation password)

Register - Server

General

Connection

Parameters

SSH Tunnel

Advanced

Post Connection SQL

Tags

Host name/address

localhost

Port

5432

Maintenance database

postgres

Username

postgres

Kerberos authentication?

☐

Password

....

Save password?

☐

Role

Service

i

?

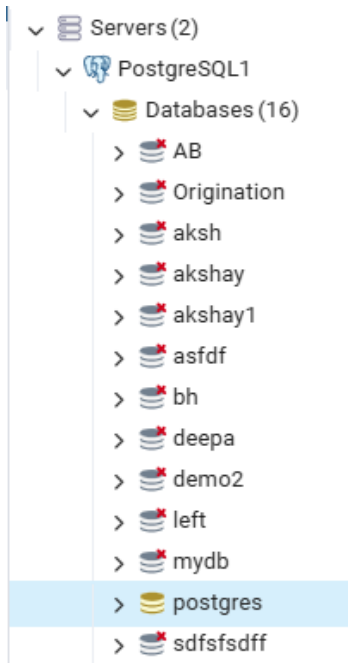
Close

Reset

Save

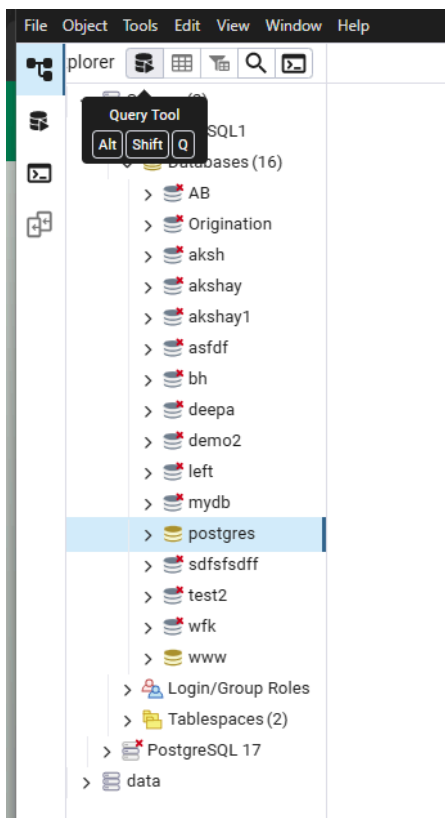
Step 5: Save and Expand Server

- Click **Save**.
- Expand the server to view the default postgres database.



Step 6: Open Query Tool

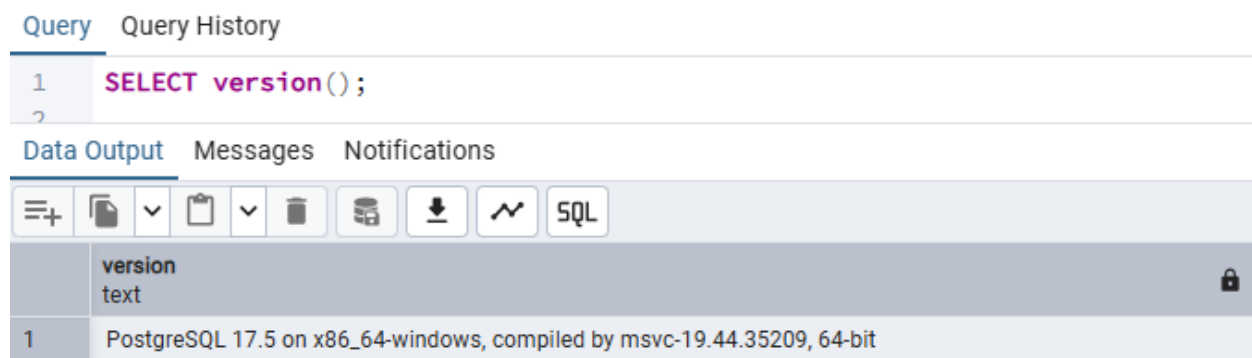
- Navigate to **Tools** → **Query Tool**, or click the Query Tool icon.



Step 7: Run SQL Command

```
SELECT version();
```

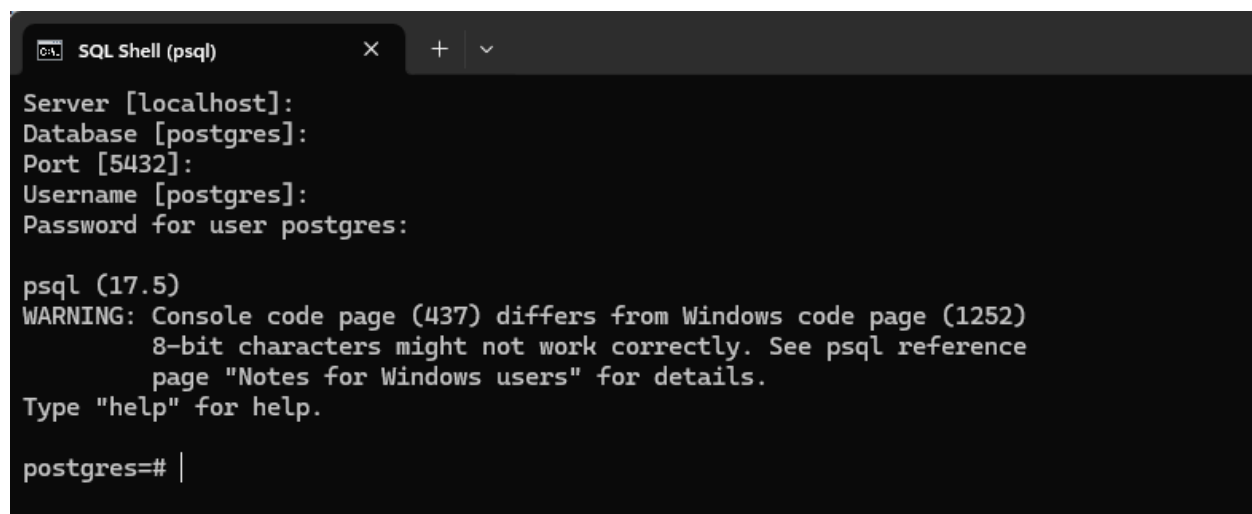
Output: Displays the PostgreSQL version.



4. Connecting with SQL Shell (psql)

Step 1: Open SQL Shell

- Search for SQL Shell (psql) and open it.
- provide the password for the user (which we created earlier) to connect the database server using the Postgres user who is a by default user in PostgreSQL



Step 2: Connect to Database

- Press **Enter** to accept default values:
 - Server: localhost
 - Database: postgres
 - Port: 5432
 - Username: postgres
- Enter your password when prompted.

Step 3: Verify Connection

- Prompt changes to:

postgres=#

Step 4: Run SQL Command

```
SELECT version();
```

Output: Displays PostgreSQL version.

```
SQL Shell (psql)
Server [localhost]:
Database [postgres]:
Port [5432]:
Username [postgres]:
Password for user postgres:

psql (17.5)
WARNING: Console code page (437) differs from Windows code page (1252)
         8-bit characters might not work correctly. See psql reference
         page "Notes for Windows users" for details.
Type "help" for help.

postgres=# SELECT version();
              version
-----
PostgreSQL 17.5 on x86_64-windows, compiled by msvc-19.44.35209, 64-bit
(1 row)

postgres=# |
```

Notes:

- Every SQL command must end with a semicolon ;.
- Pressing Enter at prompts accepts default values [].

5. Connecting from Applications

PostgreSQL can be connected from programming applications using drivers:

- **Java** → JDBC Driver
- **Python** → psycopg2 or SQLAlchemy
- **PHP** → PDO_PGSQL
- Any ODBC/JDBC-supported software

6. Summary Table

Tool	Use Case	Advantages
pgAdmin	GUI management	Beginner-friendly, visual, easy to navigate
psql	Command-line SQL execution	Direct SQL control, lightweight, powerful

Default connection parameters:

- Host = localhost
- Port = 5432
- User = postgres
- Database = postgres

PostgreSQL: Create Database

1. Introduction

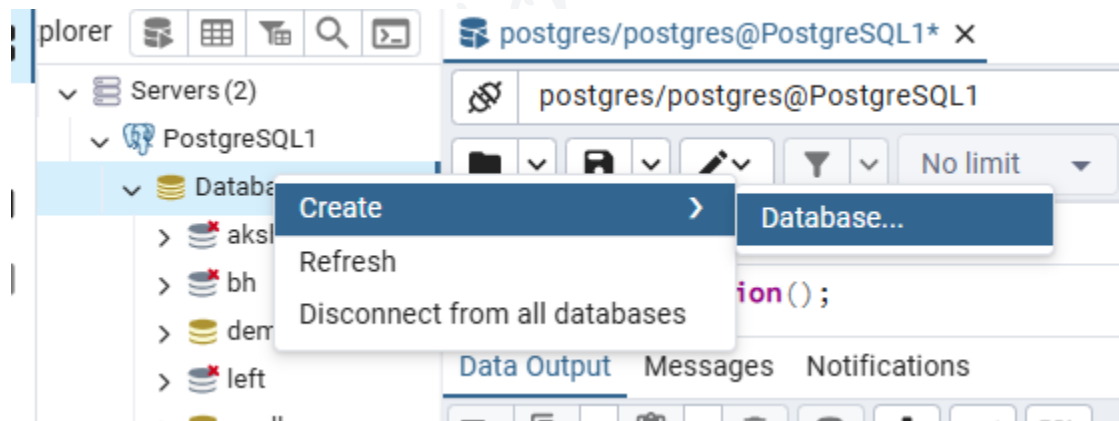
Creating a database is the first step in managing data in PostgreSQL. There are **two main ways** to create a database:

1. Using **pgAdmin (GUI)**
2. Using **SQL Shell (psql / Command Line)**

2. Create Database using pgAdmin (GUI Tool)

Step 1: Open pgAdmin

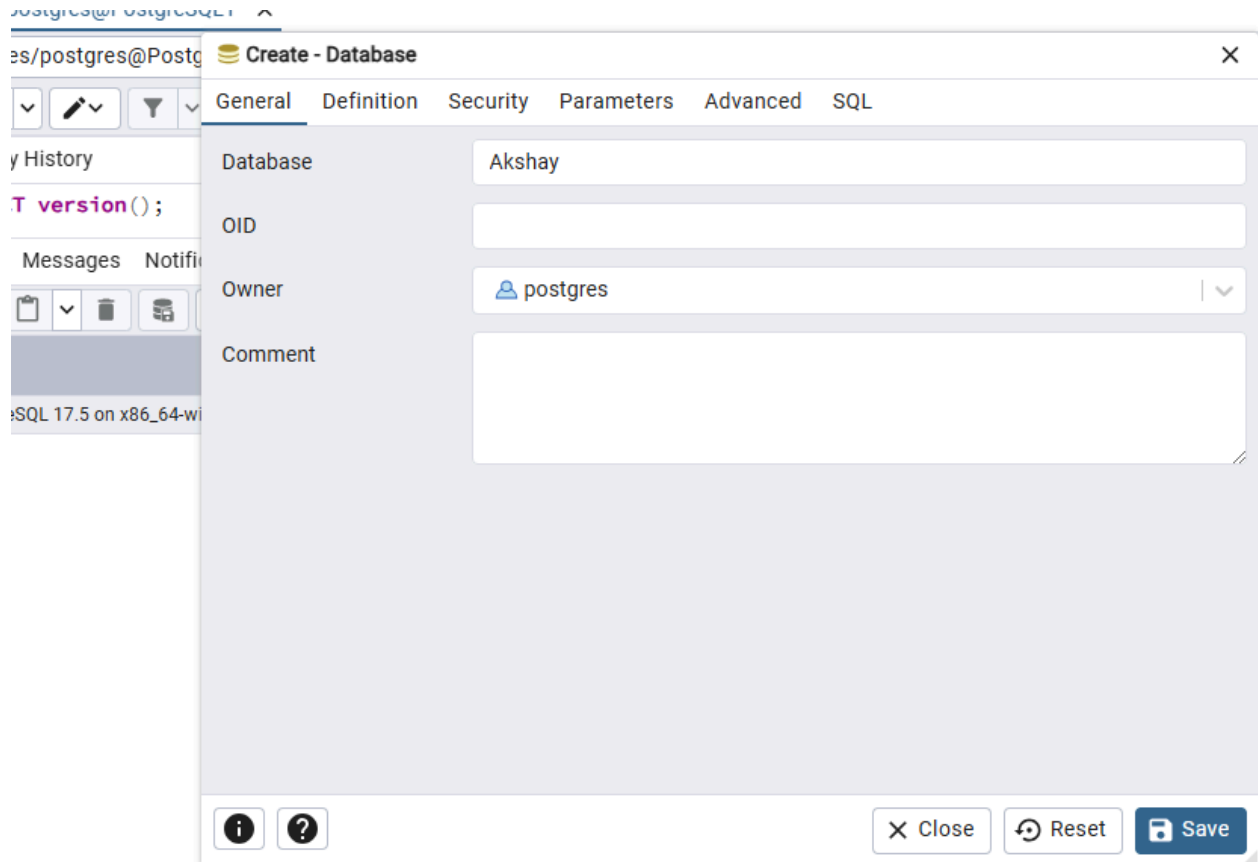
- Launch pgAdmin on your local system.
- In the **Object Browser**, right-click **Databases** → **Create** → **Database**.



Step 2: Provide Database Details

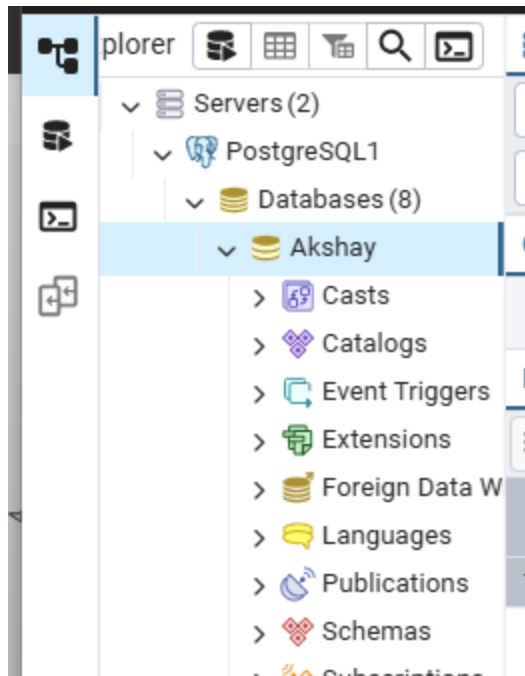
- In the Create Database window, enter:
 - **Database Name** (must be unique)

- Optional **Comment**
- Click **Save**.



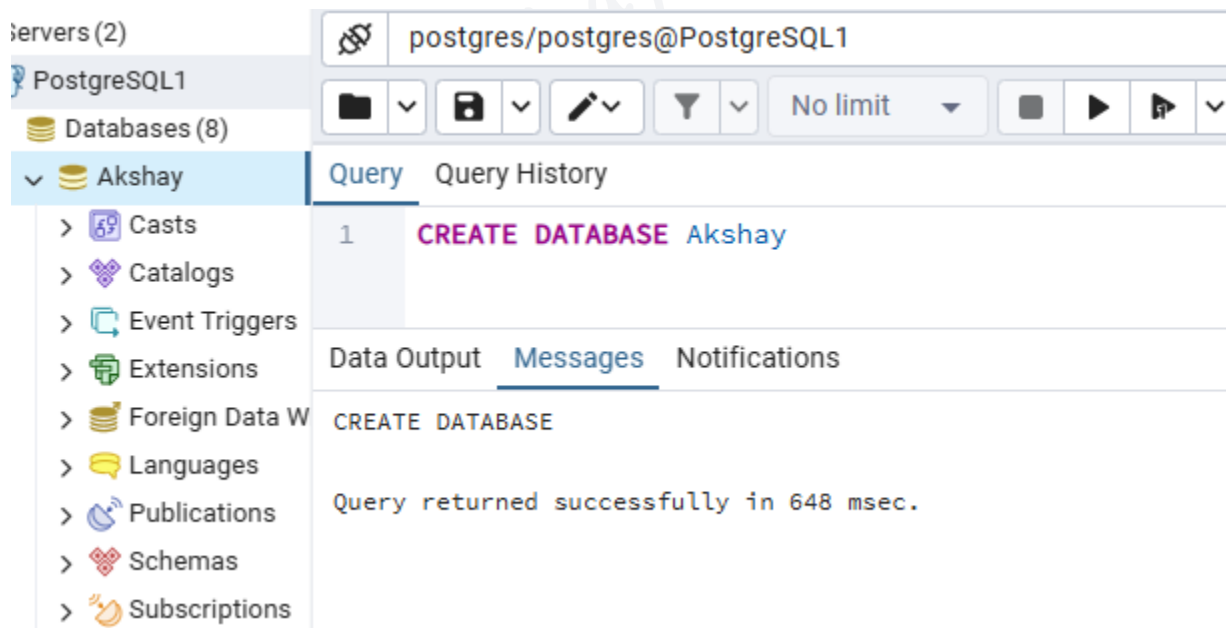
Step 3: Verify Database Creation

- The new database will appear in the Object Browser.



Step 4: View SQL Command

- pgAdmin shows the SQL used to create the database.



3. Syntax to Create Database

CREATE DATABASE db_name

OWNER = role_name

TEMPLATE = template

ENCODING = encoding

LC_COLLATE = collate

LC_CTYPE = ctype

TABLESPACE = tablespace_name

CONNECTION LIMIT = max_concurrent_connection;

Parameters

Parameter	Description
db_name	Name of the new database (must be unique).
role_name	Owner of the database (default: postgres).
template	Template database to use for creating the new database.
encoding	Character set for the database (default: UTF8).
collate	Sort order for strings (affects ORDER BY).

ctype	Character classification rules.
tablespace_name	Tablespace for storing data (default: template DB's tablespace).
max_concurrent_connection	Maximum parallel connections (default: -1, unlimited).

4. Common Errors

Error	Cause
No such file / server not accepting connections	PostgreSQL server not started.
Permission is not granted to create a database	User lacks CREATE DATABASE privileges.
Create Database command not found	PostgreSQL not installed correctly; may need to run from installation path.

5. Create Database using SQL Shell (psql / Command Line)

Step 1: Open SQL Shell (psql)

- Search for **SQL Shell (psql)** and open it.

Step 2: Connect to PostgreSQL

- Press **Enter** to accept default server, database, port, and username.
- Enter your password when prompted.

Step 3: Create Database Command

```
CREATE DATABASE Akshay;
```

Step 4: List All Databases

```
\l
```

- Displays a list of all databases on the server.

Step 5: Connect to the Database

```
\c Akshay
```

- Confirms that you are now connected to the Akshaydatabase.

Step 6: Next Steps

- After connecting, you can create tables, triggers, and other database objects.

Summary

- Databases can be created using **pgAdmin** (GUI) or **psql** (CLI).
- Ensure **database names are unique**.
- Default owner is postgres.
- Use `\l` to list all databases and `\c db_name` to connect.
- Always ensure **PostgreSQL server is running** before creating a database.

```

postgres=# \l

```

List of databases									
Name	Owner	Encoding	Locale Provider	Collate	Ctype	Locale	ICU Rules	Access privileges	
Akshay	postgres	UTF8	libc	English_United States.1252	English_United States.1252				
akshay	postgres	UTF8	libc	English_United States.1252	English_United States.1252				
bh	postgres	UTF8	libc	English_United States.1252	English_United States.1252				
demo2	postgres	UTF8	libc	English_United States.1252	English_United States.1252				
left	postgres	UTF8	libc	English_United States.1252	English_United States.1252				
mydb	postgres	UTF8	libc	English_United States.1252	English_United States.1252				
postgres	postgres	UTF8	libc	English_United States.1252	English_United States.1252				
template0	postgres	UTF8	libc	English_United States.1252	English_United States.1252				=c/postgres postgres=CtC/postgres =c/postgres postgres=CtC/postgres
template1	postgres	UTF8	libc	English_United States.1252	English_United States.1252				
test2	postgres	UTF8	libc	English_United States.1252	English_United States.1252				
www	postgres	UTF8	libc	English_United States.1252	English_United States.1252				(11 rows)

```

postgres=#

```

Step 3: Connect to Desired Database

\c Akshay

- After executing, you will see:

You are now connected to database "Akshay" as user "postgres".

Akshay=#

- The prompt changes to show the currently connected database.

```
postgres=# \c Akshay
You are now connected to database "Akshay" as user "postgres".
Akshay=# |
```

3. Select Database using pgAdmin (GUI Tool)

Step 1: Open pgAdmin

- Launch pgAdmin and navigate to the **Object Browser**.

Step 2: Select the Database

- Click on the database you want to connect (e.g., Akshay).

Step 3: Open Query Tool

- Go to **Tools** → **Query Tool**.
- A new window opens, connected to the selected database.

Step 4: Run SQL Commands

- You can now execute SQL queries on the selected database.

Summary

Method	Steps
psql / Command Line	1. Open SQL Shell → 2. Use \l to list databases → 3. Connect with \c database_name
pgAdmin / GUI	1. Open pgAdmin → 2. Click desired database → 3. Open Query Tool → 4. Run SQL commands

Notes:

- In psql, the prompt changes to indicate the current database.
- In pgAdmin, selecting a database automatically connects the Query Tool to it.

PostgreSQL: Delete / Drop Database

1. Introduction

The **DROP / DELETE command** permanently deletes a database along with all its data and files.

Use this command carefully because deletion is irreversible.

There are **three ways** to drop a database:

1. Using **pgAdmin (GUI)**
2. Using **SQL Shell (psql / Command Line)**
3. Using the **dropdb command** (OS-level command line)

2. Drop Database using pgAdmin (GUI Tool)

Step 1: Open pgAdmin

- Launch pgAdmin and locate the database you want to delete (e.g., Akshay).
- Right-click the database → Click **Delete / Drop**.

Step 2: Confirm Deletion

- A confirmation pop-up appears → Click **Yes**.

Result:

- The database is immediately removed from pgAdmin and the PostgreSQL server.

3. Drop Database using SQL Shell (psql / Command Line)

Syntax

```
DROP DATABASE [IF EXISTS] database_name;
```

Parameter	Description
IF EXISTS	Optional. Displays a warning instead of an error if the database does not exist.
database_name	Name of the database to drop.

Step-by-Step

Step 1: List Existing Databases

```
\l
```

Step 2: Drop Database Command

```
DROP DATABASE Akshay;
```

Possible Error:

ERROR: database "Akshay" is being accessed by other users

Step 3: Revoke Connections

```
REVOKE CONNECT ON DATABASE Akshay FROM public;
```

Step 4: Terminate Active Sessions

```
SELECT pg_terminate_backend(pg_stat_activity.pid)
```

```
FROM pg_stat_activity
```

```
WHERE pg_stat_activity.datname = 'Akshay';
```

Step 5: Drop the Database Again

DROP DATABASE Akshay;

Step 6: Verify Deletion

\|

- The database should no longer appear in the list.

Optional: Use IF EXISTS to avoid errors:

DROP DATABASE IF EXISTS Akshay;

4. Drop Database using dropdb Command (OS Level)

Syntax

dropdb [options] dbname

Option	Description
-e	Echo commands to the server.
-i	Prompt for confirmation.
--help	Display help information.
-h host	Hostname of the server.

-p port	Port number (default 5432).
-V	Show dropdb version.
-U username	PostgreSQL user.
-w	Skip password prompt.
maintenance db=dbname	Connect to a database to drop target database.
--if-exists	No error if database does not exist.
-W	Prompt for password.

Example

dropdb -h localhost -p 5432 -U postgres javatpoint

Password for user postgres: ****

- Drops the Akshay database using the OS command line.
- Requires ownership or superuser privileges.

Summary

Method	Steps / Notes
pgAdmin	Right-click database → Delete/Drop → Confirm.
psql / SQL Shell	DROP DATABASE [IF EXISTS] dbname; → May require revoking connections / terminating sessions.
dropdb command	OS-level command → Requires ownership or superuser → Can specify host, port, user, and password.

PostgreSQL: Role & User Management (GRANT, REVOKE)

1. Introduction

In PostgreSQL, **roles** are used to manage database access permissions.

A role can represent:

- A **user** (login account)
- A **group** (set of permissions that can be shared among users)

Key tasks include:

- Creating users/roles
- Assigning privileges with GRANT
- Removing privileges with REVOKE

2. Key Concepts

Term	Description
Role	An entity in PostgreSQL that can own database objects and have privileges.
User	A role with the LOGIN privilege.
Privileges	Rights to perform operations (CONNECT, CREATE, SELECT, INSERT, UPDATE, DELETE, ALL, etc.).

GRANT	Command used to give privileges to a role.
REVOKE	Command used to remove privileges from a role.

3. Creating Roles and Users

Syntax

CREATE ROLE role_name [WITH option...];

CREATE USER user_name [WITH option...];

Common Options

- LOGIN → Allows login (required for users).
- PASSWORD 'password' → Sets authentication password.
- CREATEDB → Allows creating databases.
- CREATEROLE → Allows creating other roles.
- SUPERUSER → Gives full access (use with caution).

Note:- pgAdmin (GUI) and psql (command line) do the same processed.

Example (psql)

Create a user with login and password

CREATE USER akshay WITH PASSWORD 'mypassword';

```
Query Query History
1 CREATE USER akshay WITH PASSWORD 'mypassword';
2
Data Output Messages Notifications
CREATE ROLE
Query returned successfully in 88 msec.
```

Create a role (no login, acts like a group)

CREATE ROLE managers;

```
Query Query History
1 CREATE ROLE managers;
Data Output Messages Notifications
CREATE ROLE
Query returned successfully in 67 msec.
```

4. Granting Privileges

Syntax

GRANT privilege [, ...]

ON object_type object_name [, ...]

TO role_name [, ...];

Common Privileges

- CONNECT → Connect to a database.
- CREATE → Create new schemas, tables, etc.

- SELECT → Read data.
- INSERT → Add data.
- UPDATE → Modify data.
- DELETE → Remove data.
- ALL PRIVILEGES → Grants all available privileges.

Examples (psql)

Allow user akshay to connect to a database

```
GRANT CONNECT ON DATABASE postgres TO akshay;
```

Query	Query History
1	GRANT CONNECT ON DATABASE postgres TO akshay;
2	

Data Output	Messages	Notifications
GRANT		
Query returned successfully in 66 msec.		

Grant SELECT and INSERT on a table

```
GRANT SELECT, INSERT ON employees TO akshay;
```

Query	Query History
1	GRANT SELECT, INSERT ON employees TO akshay;

Data Output	Messages	Notifications
GRANT		
Query returned successfully in 66 msec.		

If table doesn't exist Create it

Example employees table:

```
CREATE TABLE employees (  
    emp_id SERIAL PRIMARY KEY,  
    first_name VARCHAR(50),  
    last_name VARCHAR(50),  
    department VARCHAR(50),  
    salary NUMERIC(10,2)  
);
```

Grant all privileges on a schema

```
GRANT ALL PRIVILEGES ON SCHEMA public TO akshay;
```



The screenshot shows a database query interface with a 'Query' tab selected. The query history displays the following SQL statement:

```
1 GRANT ALL PRIVILEGES ON SCHEMA public TO akshay;  
2
```

Below the query history, the 'Messages' tab is selected, showing the output of the query:

```
GRANT
```

At the bottom, a status message indicates: 'Query returned successfully in 135 msec.'

5. Revoking Privileges

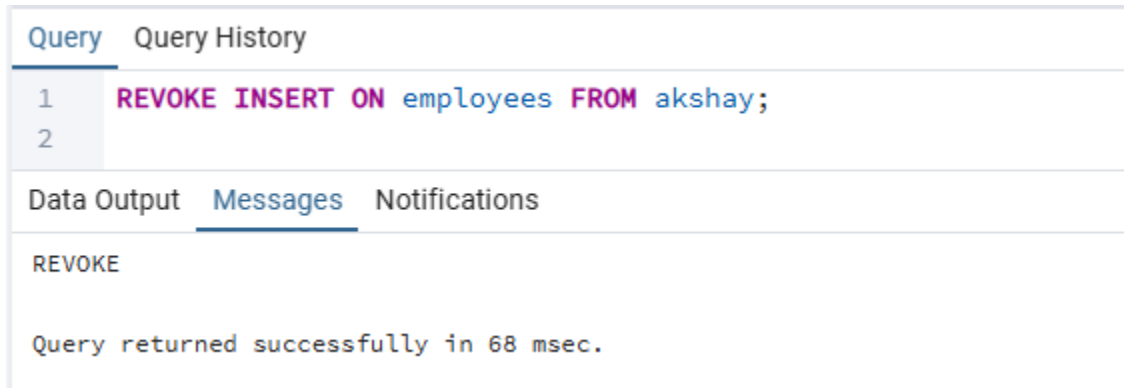
Syntax

```
REVOKE privilege [, ...]  
  
ON object_type object_name [, ...]  
  
FROM role_name [, ...];
```

Examples

Remove INSERT privilege from a user

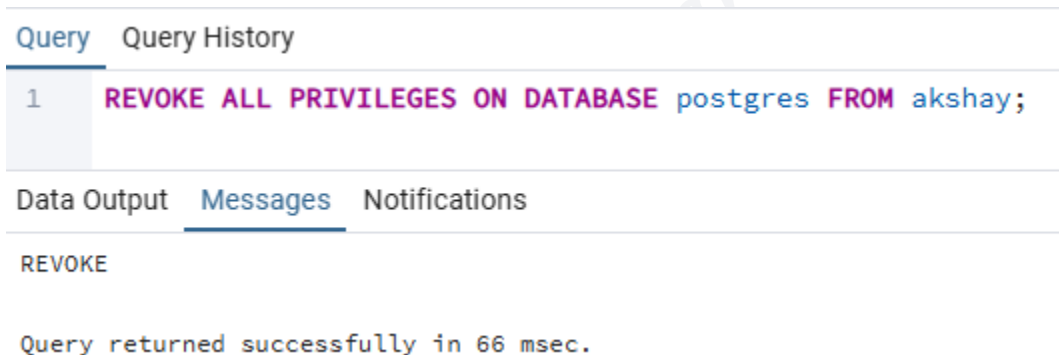
REVOKE INSERT ON employees FROM akshay;



The screenshot shows a database query interface with tabs for 'Query' and 'Query History'. The 'Query' tab is active, displaying a list of queries. Query 1 is highlighted and contains the command: **REVOKE INSERT ON employees FROM akshay;**. Below the query list, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, showing the output: 'REVOKE' and 'Query returned successfully in 68 msec.'

Remove all privileges on a database

REVOKE ALL PRIVILEGES ON DATABASE postgres FROM akshay;

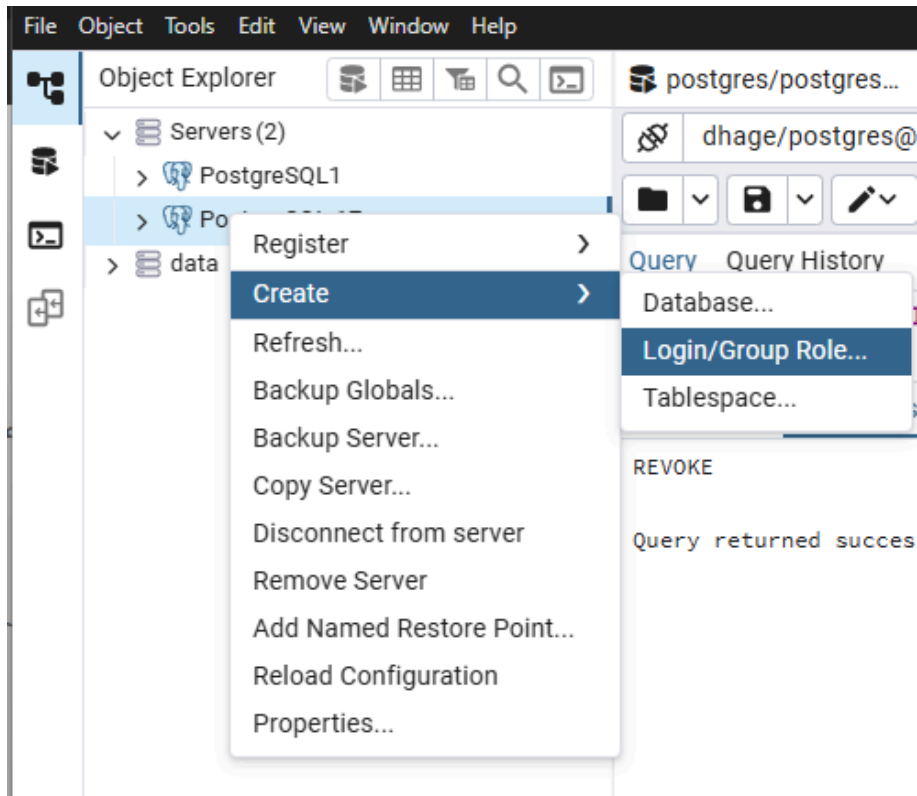


The screenshot shows a database query interface with tabs for 'Query' and 'Query History'. The 'Query' tab is active, displaying a list of queries. Query 1 is highlighted and contains the command: **REVOKE ALL PRIVILEGES ON DATABASE postgres FROM akshay;**. Below the query list, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, showing the output: 'REVOKE' and 'Query returned successfully in 66 msec.'

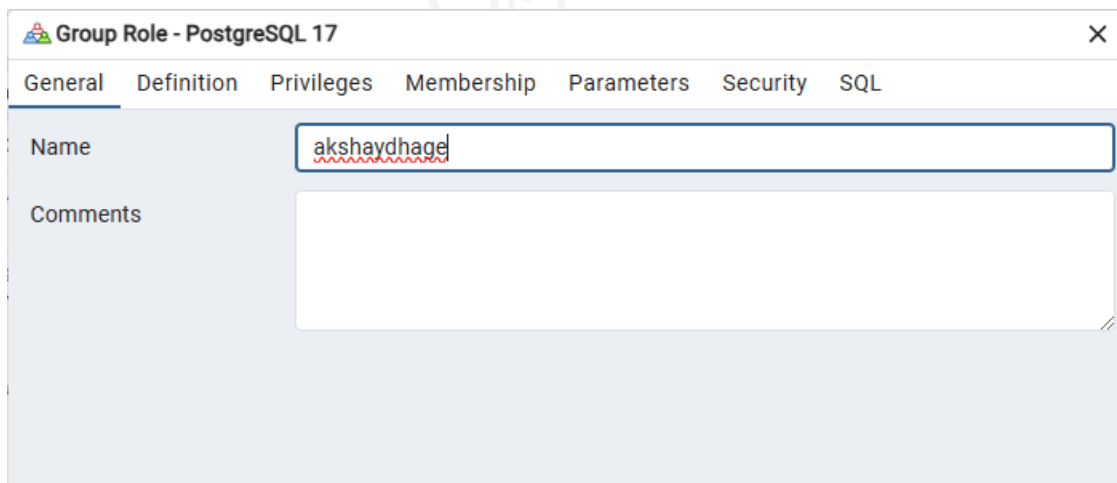
6. Managing Roles with pgAdmin (GUI Tool)

Create a User/Role

1. Open **pgAdmin** → Expand server → Right-click **Login/Group Roles** → Click **Create** → **Login/Group Role**.



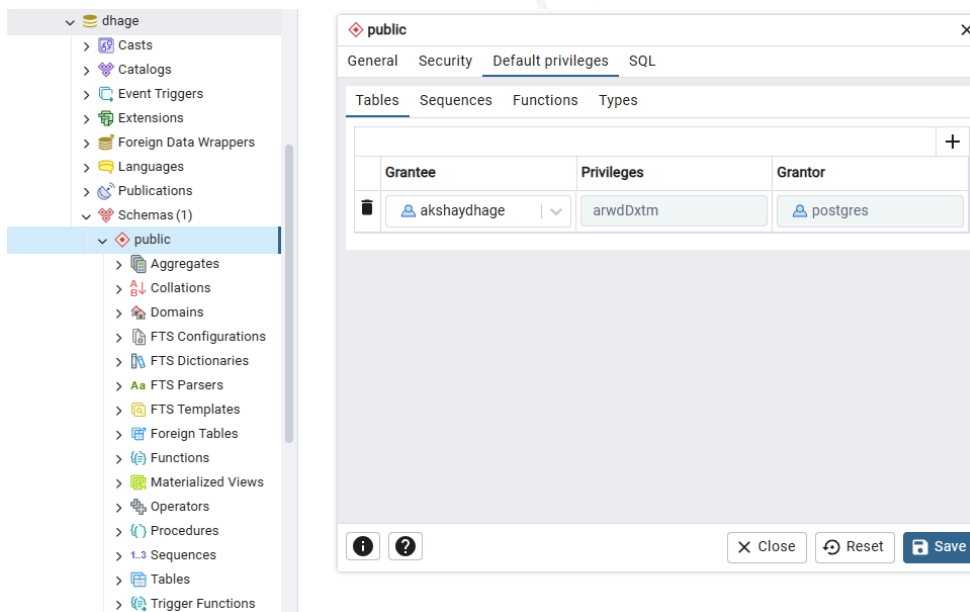
2. **General Tab:** Enter Role Name (e.g., akshaydhage).



3. **Definition Tab:** Enter a password.(akshaydhage)
4. **Privileges Tab:** Check/uncheck privileges (Can login, Create DB, etc.).
5. Save → User/Role is created.

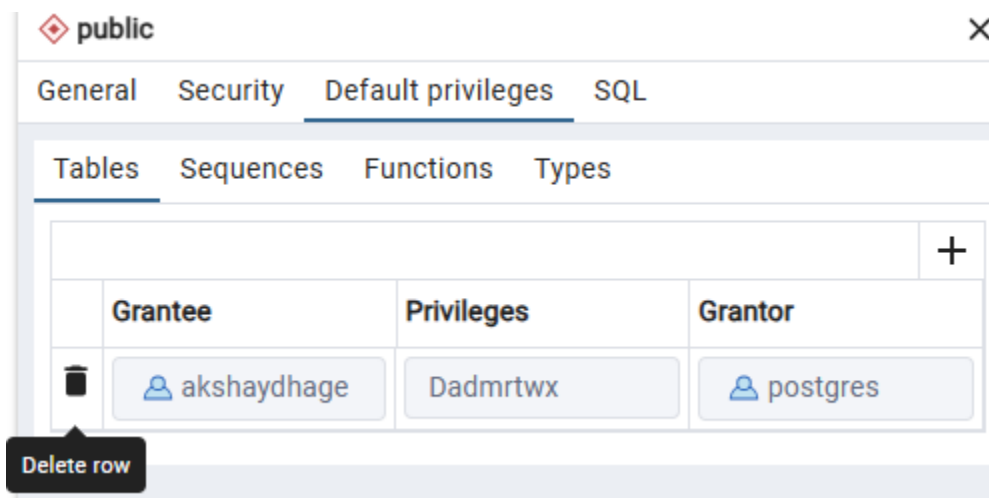
Grant Privileges

1. Navigate to the database or table
2. Expand **Schemas** → **public** → **Tables**.
3. Right-click the **table** (e.g., employees).
4. Select **Properties**.
5. Go to the **Privileges tab** (not "Default Privileges").
6. Click **+** (Add row).
7. In **Grantee**, choose your user/role (e.g., akshay).
8. Tick the permissions you want: SELECT, INSERT, UPDATE, DELETE, etc.
9. Click **Save**.



Revoke Privileges

1. Go to the same **Privileges tab**.
2. Remove the checkmarks or delete the user's permission entry.



3. Save.

7. Example: Step-by-Step

Step 1: Create a user

```
CREATE USER akshay WITH PASSWORD 'mypassword';
```

Step 2: Grant connection privilege to database

```
GRANT CONNECT ON DATABASE company TO akshay;
```

Step 3: Grant table privileges

```
GRANT SELECT, INSERT ON employees TO akshay;
```

Step 4: Revoke INSERT later

```
REVOKE INSERT ON employees FROM akshay;
```

Summary

- Roles are used to manage permissions.
- A **user** is just a role with the LOGIN attribute.
- GRANT gives privileges; REVOKE removes them.
- You can manage roles using **psql commands** or **pgAdmin GUI**.
- Common privileges: CONNECT, CREATE, SELECT, INSERT, UPDATE, DELETE, ALL.

akshay dhage

PostgreSQL: Backup & Restore (pg_dump, pg_restore)

1. Introduction

A **database backup** is a copy of your database that can be used to restore data in case of loss or damage.

PostgreSQL provides two main command-line tools for this purpose:

Tool	Description
pg_dump	Creates a backup (dump) of a PostgreSQL database.
pg_restore	Restores a backup created by pg_dump.

You can also use **pgAdmin** (the graphical tool) for backup and restore operations.

2. Backup Concepts

Term	Meaning
SQL Dump	A text file with SQL commands to recreate the database.
Custom/TAR Format	Non-text formats used for faster and partial restore.
Plain Format (.sql)	A normal readable SQL file (default).

3. Backup Using pg_dump

Syntax

`pg_dump -U username -d dbname -f filename.sql`

Common Options

Option	Description
-U	Database username
-d	Database name
-f	Output file name
-F c	Custom format backup
-F t	TAR format backup
-F p	Plain SQL script (default)
-h	Hostname
-p	Port number (default 5432)

Examples

Backup in SQL script format

```
pg_dump -U postgres -d company -f company_backup.sql
```

Backup in custom format

```
pg_dump -U postgres -F c -d company -f company_backup.dump
```

Backup in TAR format

```
pg_dump -U postgres -F t -d company -f company_backup.tar
```

4. Restore Using pg_restore

Syntax

```
pg_restore -U username -d dbname filename.dump
```

Common Options

Option	Description
-d	Target database name
-U	Database user
-c	Drop existing objects before recreating
-C	Create the database before restoring

-v	Verbose mode (shows progress)
-t	Restore only a specific table

Examples

Restore custom format backup

```
pg_restore -U postgres -d company_new company_backup.dump
```

Restore and recreate the database

```
pg_restore -U postgres -C -d postgres company_backup.dump
```

Restore only one table

```
pg_restore -U postgres -d company -t employees company_backup.dump
```

For .sql backups:

If your backup file is a plain SQL script, use psql to restore:

```
psql -U postgres -d company_new -f company_backup.sql
```

5. Backup & Restore Using pgAdmin (GUI)

Backup in pgAdmin

1. Open pgAdmin.
2. Expand Servers → Databases.
3. Right-click your database → Backup.

4. Choose:

- Format: Plain (.sql), Custom (.backup), or Tar (.tar)
- File location and name.

5. Click Backup.

Restore in pgAdmin

1. Right-click Databases → Restore.
2. Select the backup file.
3. Choose restore options (Create, Clean, etc.).
4. Click Restore.

6. Example: Step-by-Step (Your Case)

Backup Command

```
pg_dump -U postgres -d akshay -f "C:\Users\sai\Desktop\akshay_backup.sql"
```

Restore Command

```
psql -U postgres -d akshay_new -f "C:\Users\sai\Desktop\akshay_backup.sql"
```

Explanation:

- akshay → your original database
- akshay_backup.sql → the backup file
- akshay_new → restored database name

7. Common Errors and Fixes

Error 1:

'pg_dump' is not recognized as an internal or external command, operable program or batch file.

Cause:

Windows cannot find where PostgreSQL tools are installed.

Fix:

1. Go to
C:\Program Files\PostgreSQL\17\bin
2. Copy this path.
3. Add it to Environment Variables:
 - Press **Windows + R**, type sysdm.cpl, press Enter.
 - Go to **Advanced** → **Environment Variables**.
 - Under **System variables**, find **Path** → **Edit** → **New**.
 - Paste
C:\Program Files\PostgreSQL\17\bin
 - Click OK on all windows.
4. Restart Command Prompt.

Test:

```
pg_dump --version
```

If successful, it will show:

```
pg_dump (PostgreSQL) 17.x
```

Error 2:

You did not know your database name, username, or file path.

Fix:

To list databases:

```
\l
```

To check the current user:

```
\conninfo
```

To give the correct file path, use:

```
C:\Users\sai\Desktop\akshay_backup.sql
```

Error 3:

You tried running pg_dump inside SQL Shell (psql).

Fix:

pg_dump and pg_restore must be run in **Command Prompt**, not in SQL Shell.

Open Command Prompt (search "cmd" in Windows Start) and run the backup or restore commands there.

8. Summary

Tool	Function
pg_dump	Backup a PostgreSQL database
pg_restore	Restore from a dump file
psql	Restore plain .sql backup files

pgAdmin	GUI tool for backup and restore
PATH Variable	Needed to make pg_dump work in Windows

Always test your backup by restoring it into a new database.

To verify the restored database:

```
\dt          -- List tables
```

```
SELECT * FROM table_name;
```

9. Final Tips

- Always test your backup and restore process regularly.
- Store your backups in a safe and accessible location.
- Use full paths to avoid "file not found" errors.
- For regular backups, automate the pg_dump command using a batch file in Windows Task Scheduler.

PostgreSQL: Backup/Restore and Import/Export from CSV

1. Introduction

Apart from full database backups (pg_dump, pg_restore), PostgreSQL also supports **importing and exporting individual tables** using **CSV files**.

- **Export (Backup to CSV)** → Save table data into a CSV file.
- **Import (Restore from CSV)** → Load data from a CSV file into a table.

This is useful for **data migration, sharing datasets, or integrating with Excel/other applications**.

2. Export Data to CSV

Syntax (psql)

```
COPY table_name TO 'path/filename.csv' DELIMITER ',' CSV HEADER;
```

Parameters

- TO 'file' → Destination file path.
- DELIMITER ',' → Column separator (comma by default).
- CSV → Export in CSV format.
- HEADER → Include column names as the first row.

Example

Export employees table to CSV

```
COPY employees TO 'C:/postgres_backups/employees.csv' DELIMITER ',' CSV  
HEADER;
```

Result: A CSV file is created with employee data.

3. Import Data from CSV

Syntax (psql)

```
COPY table_name FROM 'path/filename.csv' DELIMITER ',' CSV HEADER;
```

Example

Import employees data from CSV

```
COPY employees FROM 'C:/postgres_backups/employees.csv' DELIMITER ',' CSV  
HEADER;
```

Requirements:

- The table must already exist with correct column structure.
- File path must be accessible to PostgreSQL server.

4. Import/Export in pgAdmin (GUI Tool)

Export

1. Right-click the **table** → Select **Export Data**.
2. Choose **Format: CSV**.
3. Set the **File path** and **Delimiter (,)**.
4. Check **Header** (if you want column names).
5. Click **OK** → Data is saved into a CSV file.

Import

1. Right-click the **table** → Select **Import/Export Data** → **Import** tab.
2. Choose **File path** of the CSV.
3. Set **Format: CSV** and ensure delimiter matches.
4. Check **Header** if CSV has column names.
5. Click **OK** → Data loads into the table.

5. Real-Life Example

Suppose you want to migrate employee data from Excel to PostgreSQL:

Step 1: Save Excel as CSV

Save the sheet as employees.csv with columns:

id,name,department,salary

1,Akshay,IT,55000

2,Raj,HR,48000

3,Sneha,Finance,60000

Step 2: Create Table

```
CREATE TABLE employees (  
    id INT PRIMARY KEY,  
    name VARCHAR(50),  
    department VARCHAR(50),  
    salary NUMERIC  
);
```

Step 3: Import CSV

```
COPY employees FROM 'C:/data/employees.csv' DELIMITER ',' CSV HEADER;
```

Step 4: Verify

```
SELECT * FROM employees;
```

Output shows imported rows.

6. Tips & Common Errors

Error	Cause	Solution
permission denied	PostgreSQL service user has no access to file path	Use path accessible by PostgreSQL or run psql with superuser
invalid input syntax	Wrong delimiter or column mismatch	Check delimiter and table structure
duplicate key value	Primary key already exists	Remove duplicates in CSV before import

7. Summary

- Use COPY TO for export and COPY FROM for import.
- Always include CSV HEADER for better readability.
- File path must be accessible to PostgreSQL.
- In pgAdmin, use **Import/Export Data** option for CSV handling.
- Useful for data migration, reporting, and integration with external apps like Excel.

akshay dhage

Download Complete SQL & PostgreSQL Notes + Practice Files

You can access the full set of **SQL/PostgreSQL notes** along with practice datasets and queries from this GitHub repository:

👉 [SQL-resources-and-tutorials by akshay-dhage](#)